

YourTranscript

A Complete Code Walkthrough

Every file, every decision, explained from first principles.

A technical companion for reading the source of the `yt-transcript` project.

Written for a first-year computer-science reader. Estimated reading time: 3–4 hours.

Stack: TypeScript · React 19 · Next.js 16 (App Router) · Tailwind CSS 4 · PostgreSQL (Neon) · Google Gemini

How to read this document

This document explains the entire `yt-transcript` codebase: the folder structure, every source file, the syntax of the languages involved, the reasons behind the main design decisions, the current limits of the application, and what changing or scaling it would require. It assumes you can program a little but does not assume you already know React, Next.js, or web servers.

The material is arranged so you can read it front to back. Earlier chapters introduce vocabulary that later chapters rely on. If a term is unfamiliar, it is either explained where it first appears or listed in the glossary at the end.

Three conventions are used throughout:

- A dark pill like `src/lib/db.ts` names a file in the repository.
- Fixed-width text like `useState` is a piece of code — a function, variable, type, or literal value.
- Indented grey blocks are excerpts of the actual source, quoted verbatim, followed by a line-by-line explanation.

On tone. This is a reference, not a sales pitch. It describes what the code does and why, without adjectives about how good it is. Where a decision has a downside, the downside is stated.

Table of contents

The application in one page

The languages and tools, and why each is here

TypeScript and JavaScript

React

Next.js and the App Router

Tailwind CSS

PostgreSQL and Neon

External services: Supadata, Gemini, YouTube, Google OAuth

The big picture: how a request flows through the system

The repository, folder by folder

Configuration files

The `lib/` layer — the plain logic

`db.ts` — the database

`cleanup.ts` — the AI transcript cleaner

`highlights.ts` — highlights, notes, progress

`google.ts` — OAuth and the YouTube feed

`wavePath.ts` — the decorative wave geometry

The API routes — the server endpoints

The React components

`HomeWaves.tsx`

`ReadingSettingsMenu.tsx`

`TranscriptReader.tsx`

The pages

`layout.tsx` and fonts

`page.tsx` — the home page

`transcribing/page.tsx` — the loading screen

`transcript/[id]/page.tsx` — the reader

`library/page.tsx`

`feed/` — the YouTube feed and watch page

Styling: `globals.css` and how Tailwind is wired

Cross-cutting concepts you now understand

Architectural decisions, restated

The limits of the current application

What scaling would require

Glossary

Syntax appendix

1 · The application in one page

YourTranscript turns a YouTube video into clean, readable text. You paste a link; the app fetches the video's captions, cleans them up with an AI model, and shows you a magazine-style transcript you can read, highlight, annotate, translate, and download.

Around that core, several features have been added:

- **A library.** Every transcript is saved to a database, so it appears in a "Saved Transcripts" list and can be reopened later from any device.
- **A reader with reading settings.** The transcript page has a theme picker (light/dark/sepia), font and size controls, column width, and on-the-fly translation to French or Spanish.
- **Highlights and notes.** You can select text to highlight it (stored in the browser) and attach a note. A dictionary lookup is available for single words or short phrases.
- **Reading-progress memory.** The reader records how far you have scrolled and returns you there next time; the library shows a percentage.
- **A YouTube feed.** If you connect a Google account, the app lists recent videos from your subscriptions, lets you send any of them to the library (which transcribes it), and offers an on-demand AI summary with the video embedded.

The whole thing is one **Next.js** web application. There is no separate backend server written in another language; the "server" parts are functions that live in the same project and run on demand. The database is hosted (Neon PostgreSQL). Three outside services do the heavy lifting the app cannot do itself: **Supadata** (fetches captions from YouTube), **Google Gemini** (the AI that cleans, translates, defines, and summarizes), and the **YouTube Data API** (lists your subscriptions). It is deployed on **Vercel**.

Persistence is split deliberately. Transcripts themselves live in the shared database so they are available anywhere. Everything personal and lightweight — highlights, notes, reading preferences, scroll progress, which feed videos you dismissed — lives in the browser's `localStorage`, tied to the device you are using. There is no login system for end users and no per-user accounts; the app currently behaves as a single-user tool (the Google connection stores one set of tokens).

Mental model. Think of the project as a set of *pages* the user sees, a set of *API endpoints* the pages call, and a small *library of plain functions* the endpoints share. Data comes to rest in two places: a Postgres table (transcripts) and the browser (everything personal).

2 · The languages and tools, and why each is here

Before walking through files, it helps to know what you are looking at and why it was chosen. Five languages/notations appear in this project, plus a handful of frameworks and services.

2.1 · TypeScript and JavaScript

JavaScript is the language web browsers run. Everything interactive on a web page ultimately becomes JavaScript. **TypeScript** is JavaScript with a type system bolted on top: you can annotate what kind of value each variable holds, and a compiler checks those annotations before the code runs. TypeScript does not run in the browser directly — it is *compiled* (translated) to plain JavaScript first. The `.ts` and `.tsx` file extensions mark TypeScript files (the `x` means the file also contains JSX, described under React).

Why TypeScript is used here: the errors it catches are the boring, common ones — passing a number where a string was expected, reading a field that might be missing, typos in property names. In a project with many small pieces talking to each other, that checking removes a whole category of bugs before the code is ever deployed. The cost is that you write more (the type annotations) and occasionally fight the compiler.

A minimal example of the difference:

```
// JavaScript: nothing stops you passing the wrong thing
function greet(name) { return "Hi " + name }

// TypeScript: `name` must be a string; the compiler enforces it
function greet(name: string): string { return "Hi " + name }
```

The `: string` after `name` is a *parameter type*. The `: string` after the parentheses is the *return type*. If you call `greet(42)`, TypeScript refuses to compile. That is the entire idea, applied consistently.

2.2 · React

React is a library for building user interfaces out of *components*. A component is a function that returns a description of what should appear on screen. React's core promise: you describe what the screen should look like *for a given set of data*, and when the data changes, React updates the screen to match. You do not manually poke at the page; you change data and let React re-render.

The description components return is written in **JSX** — HTML-like syntax embedded in the code. This is why files are `.tsx`. A tiny component:

```
function Hello({ name }: { name: string }) {
  return <p>Hello, {name}</p>
}
```

Reading it: `Hello` is a function taking one argument, an object with a `name` field of type string (that `{ name }: { name: string }` shape is destructuring plus a type, explained in the appendix). It returns JSX: a paragraph containing the text "Hello, " followed by the value of `name`. Curly braces inside JSX switch from "markup" back to "JavaScript expression."

State and hooks. Components remember things using *hooks* — special functions whose names start with `use`. The two you will see constantly:

- `useState` holds a value that can change over time; changing it re-renders the component. `const [count, setCount] = useState(0)` creates a state variable `count` starting at `0` and a function `setCount` to change it.
- `useEffect` runs code *after* the component renders — used for things that reach outside React, like fetching data, reading `localStorage`, or subscribing to browser events. It takes a function and a "dependency array"; it re-runs when any dependency changes.

You will also meet `useRef` (a mutable box that survives re-renders without causing one — often a handle to a DOM element), `useMemo` (compute a value once and reuse it unless inputs change), `useCallback` (the same idea for functions), and `useLayoutEffect` (like `useEffect` but runs before the browser paints, used when you must measure the page). React 19 is the version here.

2.3 · Next.js and the App Router

Next.js is a framework built on top of React. React alone only builds the interface that runs in the browser; it does not, by itself, give you pages with URLs, a server, or a way to run backend code. Next.js supplies all of that. This project uses Next.js 16 with its **App Router**, the newer of its two routing systems.

The App Router's central idea is **routing by folders**. Inside `src/app`, the folder structure *is* the URL structure:

- A file named `page.tsx` in a folder becomes a visitable page at that folder's path. `src/app/library/page.tsx` is the page at `/library`.
- A file named `route.ts` becomes an *API endpoint* — a URL that returns data instead of a page. `src/app/api/transcripts/route.ts` answers requests to `/api/transcripts`.
- A folder named with square brackets, like `[id]`, is a *dynamic segment* — it matches any value and hands it to the code as a parameter. `transcript/[id]/page.tsx` serves `/transcript/abc`, `/transcript/xyz`, and so on, with `id` set accordingly.
- `layout.tsx` wraps every page beneath it with shared structure (here, the `<html>` shell and fonts).

Server and client components. This is the concept that trips up most newcomers. In the App Router, components run on the *server* by default — they render to HTML before being sent to the browser. A component only runs in the browser if its file starts with the line `'use client'`. Browser-only capabilities — `useState`, `useEffect`, click handlers, reading `localStorage`, the `window` object — require a client component. Almost every page in this project starts with `'use client'` because the pages are interactive. The API `route.ts` files are pure server code; they never run in the browser.

Why a framework at all? Without Next.js you would separately set up: a bundler, a router, a Node server for the backend endpoints, and the glue between them. Next.js packages those decisions. The API secrets (database URL, Gemini key) can live safely in server-only code because that code never ships to the browser.

2.4 · Tailwind CSS

CSS is the language that styles web pages — colors, spacing, fonts, layout. Normally you write CSS in separate files with named rules. **Tailwind** is a different approach: instead of writing rules, you attach many tiny pre-made

classes directly to elements. `className="flex items-center gap-2"` means "lay children out in a row, vertically centered, with a small gap." Each of those words is one CSS declaration.

Tailwind version 4 is used here. Two Tailwind conventions appear everywhere:

- **Arbitrary values** in square brackets: `text-[1.837rem]` , `right-[9.3px]` , `bg-[#92908E]` . When no pre-made class fits, you write the exact value in brackets. This project uses many of these because its design is pixel-tuned.
- **Responsive prefixes**: `sm:text-[1.214rem]` means "apply this only on screens at least the `sm` breakpoint wide (640px)." A class with no prefix applies at all sizes; a prefixed one overrides it above the breakpoint. This is how the app adapts to phones.

Custom colors (`bg-purple` , `bg-mint` , `text-trash`) are defined once in `globals.css` and then usable as classes. Chapter 10 covers that file.

2.5 · PostgreSQL and Neon

PostgreSQL ("Postgres") is a relational database: data stored in tables of rows and columns, queried with **SQL**. This project has exactly one table, `transcripts` . **Neon** is a company that hosts Postgres databases in a "serverless" way — you do not run a database server yourself; you connect to Neon's over the network. The project talks to it through the `@neondatabase/serverless` package, which is designed to work from serverless functions (short-lived, on-demand server code) rather than a long-running server.

SQL you will see is small and readable, for example `SELECT * FROM transcripts WHERE video_id = ...` . The library used lets you write SQL inside a special tagged string; it handles safely inserting values so a malicious title cannot become an attack (this is "SQL injection" prevention, discussed with the file).

2.6 · External services

Four outside services are called over the network. Each is reached with an API key or token kept in an environment variable (a named secret configured in the deployment, never written in the code):

Service	What it does here	Secret
Supadata	Given a video ID, returns the raw caption text (the thing YouTube auto-generates).	<code>SUPADATA_API_KEY</code>
Google Gemini	The AI model. Cleans raw captions into readable prose; translates; defines words; summarizes.	<code>GEMINI_API_KEY</code>
YouTube Data API	Lists the signed-in user's subscriptions and each channel's recent uploads.	OAuth token (per user)
Google OAuth	The sign-in flow that produces the YouTube token.	<code>GOOGLE_CLIENT_ID</code> , <code>GOOGLE_CLIENT_SECRET</code>
Free Dictionary API	Dictionary definitions for single words (no key). Gemini is the fallback.	none

The database connection string (`DATABASE_URL`) is a sixth secret. Keeping all of these out of the code and in environment variables is why the backend logic must run on the server, not in the browser: the browser could not be trusted with them.

3 · The big picture: how a request flows through the system

Before the file-by-file tour, follow one complete action end to end: a user pastes a link and reads the result. Every subsystem appears in this one path.

3.1 · The transcription path, step by step

1. **Home page.** The user types a URL into the box on `/` (the file `src/app/page.tsx`) and clicks "Transcribe." The page does a light check that the text looks like a YouTube link, stores the URL in `sessionStorage` under the key `pending-url`, and navigates to `/transcribing`.
2. **Loading screen.** `src/app/transcribing/page.tsx` reads `pending-url`, shows the animated waves, and makes a network request: `POST /api/transcribe` with the URL in the body.
3. **The transcribe endpoint.** `src/app/api/transcribe/route.ts` runs on the server. It extracts the video ID from the URL, checks the database for a cached transcript, and if none exists, calls Supadata for the raw captions, groups them into paragraphs, sends them to Gemini for cleanup, fetches the title and publish date, saves everything to Postgres, and returns the segments as JSON.
4. **Back on the loading screen.** The response arrives. The page stores the transcript in `localStorage` under `transcript-<id>` so the next page can show it instantly, waits until the animation has run a minimum time, then navigates to `/transcript/<id>`.
5. **The reader.** `src/app/transcript/[id]/page.tsx` reads that `localStorage` copy synchronously so there is no second loading flash, and renders the transcript through the `TranscriptReader` component. From here the reading settings, highlights, translation, and download features take over — all described in their own sections.

Notice the two-place persistence at work: the transcript went into **Postgres** (so it will be in the library forever and on any device) and into **localStorage** (so this device shows it without a round-trip). If you open the same transcript later on a different computer, step 5 will not find a local copy and will instead fetch it from `/api/transcript/<id>`, which reads Postgres.

3.2 · Why a separate loading page at all

Caption fetching plus AI cleanup can take many seconds; a long one can approach the limit. Rather than freeze the home page, the app hands off to a dedicated screen whose only job is to wait and entertain. That screen also enforces a *minimum* visible time so that even a cached, instant response still shows the animation for a consistent moment instead of flickering past. This is a deliberate user-experience decision encoded in constants, covered in the loading-page section.

3.3 · The shape of the data

One data structure recurs everywhere: a transcript is an array of **segments**, and each segment is an object with the text and a start time in milliseconds.


```
interface Segment {  
  text: string  
  startMs: number  
}
```

After cleanup, each segment is one paragraph. The `startMs` is carried through from the caption timings (it is currently set to `0` for cleaned paragraphs, because cleanup regroups the text and the original per-word timings no longer line up — a known simplification). A saved transcript record adds the surrounding metadata:

```
interface TranscriptRecord {  
  videoId: string    // the YouTube id, also our primary key  
  url: string        // the original link  
  title: string       // fetched from YouTube  
  author: string      // the channel name  
  segments: Segment[] // the cleaned paragraphs  
  createdAt: string   // when we saved it  
  publishedAt: string | null // when the video itself came out  
}
```

Hold these two shapes in mind; most files either produce, transform, store, or display them.

4 · The repository, folder by folder

The project is a standard Next.js layout. Everything meaningful lives under `src`. The rest of the top level is configuration.

Top level

Path	Purpose
<code>package.json</code>	Lists dependencies and the <code>dev / build / start / lint</code> commands.
<code>package-lock.json</code>	An exact, machine-generated record of every installed package version. Not edited by hand.
<code>tsconfig.json</code>	TypeScript compiler settings.
<code>next.config.ts</code>	Next.js settings (essentially empty here).
<code>postcss.config.mjs</code>	Wires Tailwind into the CSS build.
<code>eslint.config.mjs</code>	Linting rules (style/error checks that run separately from compiling).
<code>vercel.json</code>	Deployment settings for Vercel; here, raising one function's time limit.
<code>next-env.d.ts</code>	Auto-generated TypeScript glue for Next.js. Never edited.

Inside `src`

Three folders, with a clear division of labor:

Folder	Contains	Runs where
<code>src/app</code>	Pages (<code>page.tsx</code>), API endpoints (<code>route.ts</code>), the root layout, and global CSS. The router reads this folder to build URLs.	Pages: browser. Routes: server.
<code>src/components</code>	Reusable pieces of interface shared by pages: the two wave renderers, the reading-settings menu, the transcript reader.	Browser.
<code>src/lib</code>	Plain logic with no interface: database access, the AI cleanup call, highlight storage, the Google/YouTube helper, wave geometry math.	Mostly server; a couple are browser utilities.

The full inventory, grouped:

```

src/
  app/
    layout.tsx          root HTML shell + fonts
    page.tsx            home page (/)
    globals.css         all global styles + Tailwind
    icon.svg, favicon.ico the tab icon
    transcribing/page.tsx loading screen (/transcribing)
    transcript/[id]/page.tsx the reader (/transcript/:id)
    library/page.tsx    saved transcripts (/library)
    feed/page.tsx       YouTube feed (/feed)
    feed/[videoId]/page.tsx watch + summary (/feed/:videoId)
  api/
    transcribe/route.ts    POST: make a transcript
    transcript/[id]/route.ts GET: one transcript from the db
    transcripts/route.ts    GET: list all transcripts
    transcripts/[id]/route.ts DELETE: remove one
    translate/route.ts     POST: translate a transcript
    define/route.ts        POST: define a word (Gemini fallback)
    summary/route.ts       POST: summarize a video
    feed/route.ts          GET: the user's YouTube feed
    auth/google/route.ts   GET: start Google sign-in
    auth/google/callback/route.ts GET: finish sign-in
    auth/google/status/route.ts GET: are we connected?
    auth/google/logout/route.ts GET: disconnect
  components/
    HomeWaves.tsx         the homepage wave animation
    ReadingSettingsMenu.tsx theme/font/size/width/language menu
    TranscriptReader.tsx   renders text; highlights, notes, dictionary
  lib/
    db.ts                 Postgres access
    cleanup.ts            raw captions -> clean paragraphs (Gemini)
    highlights.ts         localStorage: highlights, notes, progress
    google.ts             OAuth + YouTube feed assembly
    wavePath.ts           SVG wave path math

```

This is the whole application: about 3,500 lines. It is small enough to hold in your head once you have read this document.

5 · Configuration files

These files are not glamorous but they decide how everything compiles, styles, and deploys. Read them once and you rarely touch them again.

5.1 · `package.json`

```
{
  "name": "yt-transcript",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "@neondatabase/serverless": "^1.1.0",
    "next": "16.2.9",
    "react": "19.2.4",
    "react-dom": "19.2.4"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "tailwindcss": "^4",
    "typescript": "^5",
    "eslint": "^9",
    "eslint-config-next": "16.2.9",
    "@types/node": "^20", "@types/react": "^19", "@types/react-dom": "^19"
  }
}
```

Scripts are named shortcuts run with `npm run <name>`. `dev` starts a local development server with live reloading; `build` compiles the production version; `start` serves that build; `lint` runs the style/error checker.

dependencies are packages shipped as part of the running app. There are only four: the Neon database driver, Next.js, and React (the library plus its DOM renderer). That short list is itself a design fact — the project leans on the framework and writes the rest itself rather than pulling in many libraries.

devDependencies are needed only while developing or building, not at runtime: Tailwind and its PostCSS plugin, the TypeScript compiler, ESLint, and the `@types/*` packages (type definitions that teach TypeScript about Node and React). The caret `^` before a version means "this version or any compatible newer one." Exact versions (no caret) pin Next.js and React precisely.

5.2 · `tsconfig.json`

The TypeScript compiler's settings. The lines that matter for reading the code:

- `"strict": true` — turns on all the strict type checks. This is why, for example, a value that might be `null` must be handled before use.
- `"jsx": "react-jsx"` — tells the compiler to understand JSX and how to turn it into React calls.

- `"moduleResolution": "bundler"` and `"module": "esnext"` — use modern `import / export` syntax and resolve packages the way a modern bundler does.
- `"noEmit": true` — the compiler only *checks* types; it does not itself produce output files. Next.js does the actual compiling to JavaScript. TypeScript here is purely a safety net.
- `"paths": { "@/*": [".src/*"] }` — defines the `@/` import shortcut. `import { getTranscript } from '@lib/db'` means `src/lib/db`, no matter how deep the importing file is. You will see `@/` in nearly every file.

5.3 • `next.config.ts`, `postcss.config.mjs`, `eslint.config.mjs`

`next.config.ts` is effectively empty — the defaults are used. `postcss.config.mjs` registers `@tailwindcss/postcss`, which is the mechanism that turns Tailwind's classes into real CSS at build time. `eslint.config.mjs` pulls in Next.js's recommended lint rule sets (`core-web-vitals` and the TypeScript set) and tells the linter to ignore build output. None of these change how the app behaves; they configure the toolchain around it.

5.4 • `vercel.json`

```
{
  "functions": {
    "src/app/api/transcribe/route.ts": { "maxDuration": 300 }
  }
}
```

Vercel runs each API route as a serverless function with a default time limit (short — often 10 seconds on the free tier). Transcription is the one slow operation: fetching captions plus an AI pass can take a while. This file raises *that one function's* ceiling to 300 seconds (five minutes). The same limit is also declared inside the route file with `export const maxDuration = 300`; the JSON here is the deployment-level counterpart. Every other endpoint keeps the default, which is appropriate because they are fast.

6 · The `lib/` layer — the plain logic

These five files hold logic with no user interface. They are the cleanest place to learn the syntax because there is no JSX in the way. Read this chapter slowly; the patterns here (async functions, `fetch`, error handling, TypeScript interfaces) repeat in every other file.

6.1 · `src/lib/db.ts` — the database

This file is the only place that talks to Postgres. Everything else that needs a transcript goes through the functions it exports. Centralizing database access in one file is a common and deliberate choice: if the storage ever changes, only this file changes.

Setting up the connection

```
import { neon } from '@neondatabase/serverless'

const connectionString = process.env.DATABASE_URL || process.env.POSTGRES_URL
const sql = connectionString ? neon(connectionString) : null

export function dbAvailable(): boolean {
  return sql !== null
}
```

`import { neon } from ...` pulls one function, `neon`, out of the Neon package. `process.env` is how server code reads environment variables; `DATABASE_URL` is the secret connection string. The `||` ("or") means "use `DATABASE_URL`, or if that is missing, `POSTGRES_URL`" — a fallback for different hosting setups.

`const sql = connectionString ? neon(connectionString) : null` uses the **ternary operator** `condition ? a : b`, which is a compact if/else that produces a value: if there is a connection string, build a query function `sql`; otherwise set `sql` to `null`. This is defensive — the app can be built and run in environments without a database, and it simply does nothing storage-related instead of crashing. `dbAvailable()` lets callers check.

Creating the table on first use

```
let ensured = false
async function ensureTable() {
  if (!sql || ensured) return
  await sql`
    CREATE TABLE IF NOT EXISTS transcripts (
      video_id  text PRIMARY KEY,
      url       text NOT NULL,
      title     text,
      author    text,
      segments  jsonb NOT NULL,
      created_at timestampz NOT NULL DEFAULT now()
    )
  `

  await sql`ALTER TABLE transcripts ADD COLUMN IF NOT EXISTS published_at text`
  ensured = true
}
```

There is no separate database-setup step; the table is created lazily the first time it is needed. `ensured` is a module-level flag so the work happens at most once per running server instance — `if (!sql || ensured)` `return` bails out early if there is no database or the table was already ensured.

The word `await` appears here for the first time. Database queries take time (they go over the network); an `async` function can `await` such an operation, which means "pause here until this finishes, without freezing everything else." Any function using `await` must be marked `async`. This asynchronous pattern is everywhere in the project because almost everything interesting is a network call.

The SQL itself: `CREATE TABLE IF NOT EXISTS` makes the table only if it is absent. Columns are declared with types — `text`, `jsonb` (JSON stored in the database, used for the segments array), `timestamptz` (a timestamp with time zone). `video_id ... PRIMARY KEY` makes the YouTube ID the unique identifier for each row. The second statement, `ALTER TABLE ... ADD COLUMN IF NOT EXISTS published_at`, is a *migration*: the `published_at` column was added after the table already existed in production, so this line brings older databases up to date without losing data.

The backtick string after `sql` is a **tagged template literal**. The Neon library reads the SQL you write and, crucially, safely substitutes any values you interpolate — you never build SQL by gluing strings together, which is what makes "SQL injection" attacks impossible here.

Reading one transcript

```
export async function getTranscript(videoId: string): Promise<TranscriptRecord | null> {
  if (!sql) return null
  await ensureTable()
  const rows = await sql`SELECT * FROM transcripts WHERE video_id = ${videoId}`
  if (rows.length === 0) return null
  const r = rows[0]
  return {
    videoId: r.video_id,
    url: r.url,
    title: r.title,
    author: r.author,
    segments: r.segments,
    createdAt: String(r.created_at),
    publishedAt: r.published_at ?? null,
  }
}
```

The return type is `Promise<TranscriptRecord | null>`. Two things there: because the function is `async`, it returns a **Promise** — a placeholder for a value that will exist later. And `TranscriptRecord | null` is a **union type**: the result is either a record or nothing. Callers must handle both, which the strict compiler enforces.

The query interpolates `${videoId}` — the Neon tag makes this safe. If no rows come back, the function returns `null`. Otherwise it takes the first row and maps the database's `snake_case` column names (`video_id`) to the code's `camelCase` field names (`videoId`). This mapping is done by hand in one place so the rest of the code deals only in tidy JavaScript names. `r.published_at ?? null` uses the **nullish-coalescing operator** `??`: "use `r.published_at`, unless it is null or undefined, in which case use `null`."

Saving a transcript

```
export async function saveTranscript(rec: {
  videoId: string; url: string; title: string; author: string
  segments: Segment[]; publishedAt?: string | null
}): Promise<void> {
  if (!sql) return
  await ensureTable()
  await sql`
    INSERT INTO transcripts (video_id, url, title, author, segments, published_at)
    VALUES (${rec.videoId}, ${rec.url}, ${rec.title}, ${rec.author},
      ${JSON.stringify(rec.segments)}::jsonb, ${rec.publishedAt ?? null})
    ON CONFLICT (video_id) DO UPDATE
    SET url = EXCLUDED.url, title = EXCLUDED.title, author = EXCLUDED.author,
      segments = EXCLUDED.segments,
      published_at = COALESCE(EXCLUDED.published_at, transcripts.published_at)
  `
}
```

The parameter type is written inline: an object with these fields, where `publishedAt?` has a question mark meaning it is *optional*. Return type `Promise<void>` means the function does its work and resolves with no value.

`JSON.stringify(rec.segments)` converts the segments array into a JSON string, and `::jsonb` casts it into Postgres's JSON type. The `ON CONFLICT (video_id) DO UPDATE` clause is an "upsert": if a row with this video ID already exists, update it instead of failing. That is why re-transcribing a video overwrites cleanly. `COALESCE(EXCLUDED.published_at, transcripts.published_at)` keeps an existing publish date if the new save happens not to have one — it never overwrites good data with nothing.

Delete and list

```
export async function deleteTranscript(videoId: string): Promise<void> {
  if (!sql) return
  await ensureTable()
  await sql`DELETE FROM transcripts WHERE video_id = ${videoId}`
}

export async function listTranscripts(): Promise<TranscriptSummary[]> {
  if (!sql) return []
  await ensureTable()
  const rows = await sql`
    SELECT video_id, url, title, author, created_at, published_at
    FROM transcripts ORDER BY created_at DESC
  `
  return rows.map((r) => ({ /* map columns to a summary object */ }))
}
```

`listTranscripts` selects a lighter set of columns (not the full segments — the library only needs titles and dates) and orders newest-first with `ORDER BY created_at DESC`. `rows.map(...)` transforms every database row into a clean summary object; `.map` is the standard way to turn one array into another by applying a function to each element. The parentheses around the returned object `{ ... }` are required so the arrow function is not mistaken for a function body.

What to take from this file. A single module owns all storage; every function guards against a missing database; column names are translated at the boundary; and query values are always interpolated through

the safe Neon tag. These are the habits worth copying.

6.2 · `src/lib/cleanup.ts` — the AI transcript cleaner

This file turns YouTube's messy auto-captions into readable paragraphs by asking Gemini to rewrite them under strict rules. It is the clearest example in the project of "prompt engineering" — most of the file is an instruction string.

The instruction (system prompt)

A long constant `SYSTEM_PROMPT` tells the model exactly what to do and, just as importantly, what *not* to do. Its rules, in plain terms:

- If the whole transcript arrives in ALL CAPS (YouTube sometimes does this), rewrite it in normal sentence case, capitalizing only real proper nouns, sentence starts, "I", and acronyms.
- Fix misheard proper nouns from context, and apply a fixed list of known mistranscriptions (for example "Palanteer" → "Palantir", "Handes" → "Andes"). This list grows as the user reports new ones.
- Remove the duplicated preview snippet YouTube puts at the very start; remove the censor tokens; thin out filler words by roughly half but not entirely, preserving the speaker's natural rhythm.
- Keep meaningful repetition (emphasis, idioms, laughter) but remove accidental stutters — a nuanced instruction with many examples so the model does not flatten the speech.
- Break the text into paragraphs by speaker turn or by distinct argument. Do **not** prefix paragraphs with speaker names unless the text states the speaker unambiguously and the model is completely certain. Never invent names or use generic labels like "Host."
- Never summarize or drop content; output only the cleaned transcript with blank lines between paragraphs and no markdown.

Each of these rules exists because an earlier version misbehaved in that specific way. The prompt is, in effect, a written record of every correction the project's author has made — which is why it reads like a list of grievances against YouTube's captions.

The call to Gemini

```

const MODEL = 'gemini-2.5-flash'
const ENDPOINT =
`https://generativelanguage.googleapis.com/v1beta/models/${MODEL}:generateContent`

export async function cleanupTranscript(rawText: string): Promise<Segment[] | null> {
  const apiKey = process.env.GEMINI_API_KEY
  if (!apiKey) return null

  let full = ''
  try {
    const res = await fetch(`${ENDPOINT}?key=${apiKey}`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        system_instruction: { parts: [{ text: SYSTEM_PROMPT }] },
        contents: [{ role: 'user', parts: [{ text: rawText }] }],
        generationConfig: { maxOutputTokens: 65536, temperature: 0.2 },
      }),
    })
    if (!res.ok) return null
    const data = await res.json()
    const parts = data?.candidates?.[0]?.content?.parts ?? []
    for (const part of parts) {
      if (typeof part?.text === 'string') full += part.text
    }
  } catch {
    return null
  }
  // ...split into paragraphs...
}

```

`fetch` is the built-in way to make an HTTP request. Here it is a `POST` to Gemini's endpoint, with the API key in the query string, a header declaring the body is JSON, and a body built by `JSON.stringify` from a nested object describing the request. `temperature: 0.2` asks the model to be nearly deterministic — low creativity — because this is a faithful rewrite, not brainstorming. `maxOutputTokens: 65536` allows a long transcript to come back whole.

The `try { ... } catch { ... }` block is error handling: if anything inside `try` throws (network failure, bad JSON), execution jumps to `catch`, which returns `null`. The `?.` in `data?.candidates?.[0]?.content?.parts` is **optional chaining**: read each step only if the previous one exists, otherwise produce `undefined` instead of crashing. This matters because the model's response shape is nested and any level could be missing. The `?? []` then substitutes an empty array so the following loop is safe.

Turning the text back into segments

```

const paragraphs = full
  .split(/\n\s*\n/)
  .map((p) => p.replace(/\s+/g, ' ').trim())
  .filter(Boolean)

if (paragraphs.length === 0) return null
return paragraphs.map((text) => ({ text, startMs: 0 }))

```

The model returns one long string with blank lines between paragraphs. `.split(/\n\s*\n/)` cuts it wherever a blank line appears (that is a **regular expression** matching a newline, optional whitespace, and another newline).

`.map(p => p.replace(/\s+/g, ' ').trim())` collapses any run of whitespace inside a paragraph into single spaces and trims the ends. `.filter(Boolean)` drops empty paragraphs (empty strings are "falsy," so `Boolean` filters them out). Finally each surviving paragraph becomes a `Segment` with `startMs: 0`.

Why single-space normalization matters. The highlight feature stores selections as character offsets into a paragraph's text. Guaranteeing exactly one space between words makes those offsets stable, so a highlight lands on the same characters every time. A small cleanup rule here quietly enables a feature two files away.

The fallback

If Gemini is unavailable or fails, `cleanupTranscript` returns `null`. The transcribe route treats `null` as "use the plain, heuristically grouped segments instead." The app still works without the AI; it just produces rougher text. Designing the AI step to be optional is a deliberate resilience choice.

6.3 • `src/lib/highlights.ts` — highlights, notes, progress

This file is the browser-side counterpart to `db.ts`: it owns everything stored in `localStorage`. Unlike the database module, it runs in the browser, so it guards every access with a check that `window` exists (the same code is prepared on the server during rendering, where `localStorage` is absent).

The Highlight shape and the storage key

```
export interface Highlight {
  id: string
  paragraph: number // which segment
  start: number     // character offset within the paragraph
  end: number       // exclusive
  text: string       // the highlighted substring
  note: string
  createdAt: number
}

const keyFor = (transcriptId: string, lang: string) =>
  `highlights-${transcriptId}-${lang}`
```

A highlight records which paragraph it is in and the `start` / `end` character positions inside that paragraph's text, plus a copy of the highlighted text and an optional note. The storage key includes the **language**: because translating a transcript changes the characters, offsets from the English version would be meaningless in French, so each language keeps its own set of highlights under its own key.

Load and save

```

export function loadHighlights(transcriptId: string, lang: string): Highlight[] {
  if (typeof window === 'undefined') return []
  try {
    const raw = localStorage.getItem(keyFor(transcriptId, lang))
    if (!raw) return []
    const parsed = JSON.parse(raw)
    return Array.isArray(parsed) ? parsed : []
  } catch { return [] }
}

export function saveHighlights(transcriptId: string, lang: string, highlights:
Highlight[]) {
  if (typeof window === 'undefined') return
  try { localStorage.setItem(keyFor(transcriptId, lang), JSON.stringify(highlights)) }
catch {}
}

```

`localStorage` only stores strings, so objects are serialized with `JSON.stringify` on the way in and `JSON.parse` on the way out. Both functions wrap their work in `try/catch` and return a safe default, because `localStorage` can be disabled or full and parsing could fail on corrupted data. `typeof window === 'undefined'` is the standard test for "am I running on the server"; if so, there is nothing to read, so it returns an empty array.

Counting for the library, and clearing on delete

```

export function countHighlights(transcriptId: string): { highlights: number; notes: number
} {
  // scan every localStorage key beginning with `highlights-<id>-`
  // sum total highlights and how many carry a non-empty note
}

export function clearLocalTranscript(transcriptId: string) {
  // remove `transcript-<id>` and every `highlights-<id>-*` key
}

```

`countHighlights` walks all storage keys (using `localStorage.key(i)` in a loop) and adds up highlights and notes across every language for that transcript — this is what produces the two counts shown on each library row. `clearLocalTranscript` is called when a transcript is deleted, so the local cache and its highlights do not linger after the database row is gone.

The offset helper

```

export function charOffsetWithin(root: HTMLElement, node: Node, offset: number): number {
  const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT)
  let count = 0
  let n: Node | null = walker.nextNode()
  while (n) {
    if (n === node) return count + offset
    count += n.textContent?.length ?? 0
    n = walker.nextNode()
  }
  return count
}

```

This translates a browser text selection into a character offset. When you select text, the browser reports the selection as a node and an offset within that node — but a paragraph may be split into several text pieces (because a previous highlight wrapped part of it in its own element). A `TreeWalker` visits every text node in order; the function adds up their lengths until it reaches the node the selection started in, then adds the offset. The result is a single number: the position within the paragraph's full text, independent of how the text is currently split into elements. The reader component uses this to store stable highlight ranges.

Reading progress

```
export function loadProgress(transcriptId: string): number { /* read integer 0-100 */ }
export function saveProgress(transcriptId: string, percent: number) { /* clamp and store */ }
```

Two small functions store a single integer per transcript: the percentage scrolled. `saveProgress` clamps the value into the 0–100 range and rounds it before storing. The reader writes this on every scroll; the library reads it to show the percentage and the reader reads it to restore your position.

6.4 · `src/lib/google.ts` — OAuth and the YouTube feed

This is the most involved `lib` file. It does two jobs: the OAuth sign-in dance with Google, and assembling a "subscription feed" from the YouTube Data API. It deliberately uses only `fetch` — no Google client library — to avoid an extra dependency.

Is it configured?

```
export function googleConfigured(): boolean {
  return !!(process.env.GOOGLE_CLIENT_ID && process.env.GOOGLE_CLIENT_SECRET)
}
```

`!!(...)` converts a value to a strict `true / false`. This returns `true` only if both Google secrets are present. The feed UI uses it to show a "not configured yet" message instead of a broken sign-in when the deployment has no Google credentials.

The three OAuth steps

OAuth is a standard three-step handshake, and this file implements each as a function:

```
export function authUrl(redirectUri: string, state: string): string {
  const params = new URLSearchParams({
    client_id: process.env.GOOGLE_CLIENT_ID ?? '',
    redirect_uri: redirectUri, response_type: 'code',
    scope: 'https://www.googleapis.com/auth/youtube.readonly',
    access_type: 'offline', prompt: 'consent', state,
  })
  return `https://accounts.google.com/o/oauth2/v2/auth?${params}`
}
```

Step one, `authUrl`, builds the Google sign-in link. `URLSearchParams` encodes the query parameters safely. `scope` asks only for read-only access to YouTube. `access_type: 'offline'` is what makes Google return a long-lived *refresh token*; `prompt: 'consent'` forces the consent screen so the refresh token is reliably issued. `state` is a random value used to detect tampering when Google redirects back.

```
export async function exchangeCode(code, redirectUri) { /* POST to /token,
grant_type=authorization_code */ }
export async function refreshAccessToken(refreshToken): Promise<string | null> { /* POST,
grant_type=refresh_token */ }
```

Step two, `exchangeCode`, trades the one-time `code` Google sends back for tokens (an access token and a refresh token). **Step three**, `refreshAccessToken`, uses the long-lived refresh token to get a fresh short-lived access token whenever the feed is loaded. Only the refresh token is stored (in a cookie); access tokens are obtained on demand and never persisted.

Assembling the feed

YouTube offers no single "my subscription feed" endpoint, so the feed is built by hand. `getFeed(token)` performs a sequence of calls, each through a small helper `yt(path, token)` that attaches the access token and parses the JSON:

1. List the user's subscriptions (paginating up to about 100 channels).
2. For each channel, look up its "uploads" playlist ID (channels are queried 50 at a time, the API's batch limit).
3. For each uploads playlist, fetch the recent items, keeping only those published within the last 30 days.
4. Collect the candidate video IDs and fetch their durations in batches of 50.
5. Drop "Shorts" — videos 180 seconds or shorter, or with "#shorts" in the title — then sort everything newest-first.

```
const isShort = (v: FeedVideo) => {
  const dur = durationById[v.videoId] ?? 0
  return (dur > 0 && dur <= 180) || /#shorts?\b/i.test(v.title)
}
return candidates.filter((v) => !isShort(v))
  .sort((a, b) => +new Date(b.publishedAt) - +new Date(a.publishedAt))
```

The date parsing `+new Date(...)` turns a date string into a number of milliseconds so two dates can be subtracted; sorting by `b - a` yields descending (newest first) order. `isShort` combines a duration test with a title pattern. The 30-day window, the batching, and the caps all exist to keep within YouTube's daily API quota — a constraint discussed in the scaling chapter.

Several calls run in parallel with `await Promise.all(uploads.map(async (u) => { ... })))`.

`Promise.all` starts many asynchronous operations at once and waits for all of them, rather than doing them one at a time — important when there are dozens of channels to query.

6.5 • `src/lib/wavePath.ts` — the decorative wave geometry

This file is pure math and produces no data or network calls: it generates the **SVG path** strings that draw the calligraphic "waves" on the home and loading pages. SVG is a vector-graphics format where a shape is described by a string of drawing commands.

```

export function makeWaveOutline(seed: number, maxHalfWidth: number) {
  const cycles = 1
  const amp = 4.8 + (seed % 3) // amplitude varies slightly per wave
  const mid = VIEW_H / 2
  const N = 48
  const pts = []
  for (let i = 0; i <= N; i++) {
    const t = i / N
    pts.push({ x: t * VIEW_W, y: mid + amp * Math.sin(2 * Math.PI * cycles * t) })
  }
  // ...offset a tapered ribbon above and below the centerline...
}

```

The function samples a sine wave at 48 points along a fixed viewbox to get a gently curved centerline (`Math.sin` produces the up-and-down shape; `amp` is how tall, varied per `seed` so no two waves are identical). It then walks that centerline and pushes each point outward by a half-width that is zero at the ends and largest in the middle (`maxHalfWidth * Math.sin(Math.PI * t)`), producing a filled ribbon that tapers to points — the brush-stroke look. The result is a single `d="..."` string of `M` (move) and `L` (line) commands closed with `Z`.

The file also exports `WAVE_SHADOWS` (a lookup from each wave color to a lighter version used as its drop shadow) and constants for the viewbox size and a pixels-per-centimeter factor. The two page components import these and animate many copies with CSS. This is the one corner of the project that is closer to graphics programming than to web plumbing, and it is fully self-contained — nothing outside the two wave components depends on it.

7 · The API routes — the server endpoints

Every file named `route.ts` under `src/app/api` is a server endpoint. Pages in the browser call these with `fetch`; the endpoints do the work that needs secrets or the database, then return JSON. A route file exports functions named after HTTP methods — `GET`, `POST`, `DELETE` — and Next.js calls the matching one.

7.1 · `api/transcribe/route.ts` — the centerpiece

This is the busiest endpoint and the heart of the app. It accepts a URL and returns a finished transcript. Read it as five stages.

Stage 1 — validate and extract the ID

```
export const maxDuration = 300

function extractVideoId(url: string): string | null {
  try {
    const u = new URL(url)
    if (u.hostname === 'youtu.be') return u.pathname.slice(1).split('?')[0]
    return u.searchParams.get('v')
  } catch { return null }
}

export async function POST(req: Request) {
  const { url } = await req.json()
  if (!url) return NextResponse.json({ error: 'No URL provided' }, { status: 400 })
  const videoId = extractVideoId(url)
  if (!videoId) return NextResponse.json({ error: 'Invalid YouTube URL.' }, { status: 400 })
}
```

`export const maxDuration = 300` is the in-code twin of the `vercel.json` setting; this function may run up to five minutes. `extractVideoId` handles both YouTube URL shapes — the `youtu.be/ID` short form and the `youtube.com/watch?v=ID` long form — using the built-in `URL` parser rather than fragile string cutting. `const { url } = await req.json()` reads and destructures the request body. Failures return a JSON error with an HTTP **status code**: `400` means "bad request." Status codes are how the caller knows success (200-range) from failure (400- and 500-range).

Stage 2 — the cache

```
const cached = await getTranscript(videoId)
if (cached) {
  return NextResponse.json({ id: videoId, segments: cached.segments, title: cached.title,
    cached: true })
}
```

Before spending money on Supadata and Gemini, the route checks the database. If this video was transcribed before, it returns the stored version immediately. This is the single most important cost- and speed-saving decision in the app: transcription happens once per video, ever, across all users.

Stage 3 — fetch the captions


```
const res = await fetch(
  `https://api.supadata.ai/v1/youtube/transcript?videoId=${videoId}&lang=en`,
  { headers: { 'x-api-key': apiKey } }
)
if (!res.ok) { /* return a 422 "no transcript available" with the reason */ }
const data = await res.json()
raw = data.content ?? []
```

Supadata is asked for the English captions. If it fails, the route returns status 422 ("unprocessable") with a human-readable reason — this is what surfaces as "No transcript available for this video" when captions are disabled. The raw result is an array of caption fragments, each with text, an offset, and a duration.

Stage 4 — group into rough paragraphs

Before the AI sees the text, the route does a first, mechanical grouping. Two strategies:

- If the captions contain `>>` markers (YouTube's convention for a new speaker), a new block starts at each marker.
- Otherwise, blocks are split on time gaps: when more than two seconds pass between fragments, a new paragraph begins.

A second pass then merges fragments that begin with a lowercase letter into the previous block, since a lowercase start usually means a continued sentence rather than a new speaker. This heuristic grouping is a safety net: if the AI step is skipped or fails, the transcript is still broken into reasonable paragraphs.

Stage 5 — clean, look up metadata, save, return

```
const rawText = raw.map((item) => item.text).join(' ')
const cleaned = await cleanupTranscript(rawText)
const finalSegments = cleaned ?? merged

const [{ title, author }, publishedAt] = await Promise.all([
  fetchTitle(videoId), fetchPublishDate(videoId),
])
await saveTranscript({ videoId, url, title, author, segments: finalSegments, publishedAt })
return NextResponse.json({ id: videoId, segments: finalSegments, title })
```

The full caption text is joined into one string and passed to `cleanupTranscript` (the file from 6.2). `const finalSegments = cleaned ?? merged` is the fallback in action: use the AI-cleaned paragraphs, or the mechanically grouped ones if cleanup returned `null`. Title and channel come from YouTube's keyless **oEmbed** endpoint; the publish date is scraped from the public watch page (there is no keyless API for it, so `fetchPublishDate` pulls it out of the page's HTML with a regular expression). Both metadata calls run together via `Promise.all`. Finally the record is saved and the segments are returned to the loading page.

Reading this route teaches the whole backend style. Validate input; check the cache; call an external service with a key; transform; store; return JSON with a status code. Every other endpoint is a smaller version of this pattern.

7.2 · The transcript CRUD routes

Three tiny endpoints wrap the database functions. "CRUD" is create/read/update/delete; these cover read, list, and delete (create happens inside `transcribe`).

- `api/transcript/[id]/route.ts` — `GET`. Returns one transcript by ID from the database (used when opening a transcript on a device that has no local copy). Note the dynamic segment: `{ params }: { params: Promise<{ id: string }> }` and `const { id } = await params` — in this Next.js version the route parameters arrive as a Promise and must be awaited.
- `api/transcripts/route.ts` — `GET`. Returns the whole list for the library. Four lines.
- `api/transcripts/[id]/route.ts` — `DELETE`. Removes one transcript. The first parameter is named `_req` with a leading underscore, a convention meaning "required by the signature but unused."

7.3 • `api/translate/route.ts`

Given a title and paragraphs and a target language (French or Spanish), this asks Gemini to translate them. The interesting part is how it keeps paragraphs aligned. It joins all chunks with a rare delimiter, `\n@@@\n`, and instructs the model to keep exactly the same number of chunks separated by that same delimiter. On the way back it splits on the delimiter and checks the count:

```
if (out.length !== chunks.length) {  
  return NextResponse.json({ error: 'Translation misaligned.' }, { status: 502 })  
}
```

If the counts do not match, it fails rather than returning misaligned text — because a paragraph mismatch would put highlights on the wrong words. This is a small example of a recurring principle: when correctness cannot be guaranteed, fail loudly instead of returning something subtly wrong.

7.4 • `api/define/route.ts` and `api/summary/route.ts`

define is the dictionary fallback. The reader first tries the free, keyless dictionary API directly from the browser; only when that returns nothing (common for non-English words and for phrases) does it call this route, which asks Gemini for a one-line definition in the target language, formatted as `part-of-speech ||| definition` and split apart on the client.

summary powers the "Ask AI" button on the feed watch page. It gathers the video's caption text — reusing a saved transcript if one exists, otherwise fetching fresh from Supadata — and asks Gemini for a one-line TL;DR followed by five to eight bullet points. If the video has no captions, it returns an empty summary and the page says so. Both routes follow the same call-Gemini-and-parse structure as `cleanup`.

7.5 • `api/feed/route.ts` and the auth routes

feed reads the refresh token from the request cookie, exchanges it for a fresh access token via `refreshAccessToken`, calls `getFeed`, and returns the list. No cookie means status `401` ("unauthorized"), which the feed page reads as "not connected."

The four **auth/google** routes implement the sign-in flow described in 6.4:

- `route.ts` (GET) — generates a random `state` , stores it in a short-lived cookie, and redirects the browser to Google's consent screen.
- `callback/route.ts` (GET) — Google returns here with a code; the route checks the `state` matches (anti-tampering), exchanges the code for tokens, stores the refresh token in an `httpOnly` cookie (unreadable by JavaScript, so it cannot be stolen by a script), and redirects to `/feed` .
- `status/route.ts` (GET) — reports whether Google is configured and whether a token cookie is present, so the page can pick which state to show.
- `logout/route.ts` (GET) — deletes the token cookie.

Storing the token server-side in an `httpOnly` cookie rather than in `localStorage` is a security decision: page scripts never see it, which limits the damage a malicious script could do. It is also why the feed is currently single-user — there is one cookie per browser, not a per-account token store.

8 · The React components

Three components in `src/components` are shared by pages. The first two are self-contained; the third, the transcript reader, is the most intricate client code in the project and rewards careful reading.

8.1 · `HomeWaves.tsx`

A client component (`'use client'`) that renders the animated background on the home page. It builds a list of wave descriptions once with `useMemo` — six rows, fourteen per row, each with a pseudo-random size, vertical position, animation delay and duration derived from a running `seed` number — then filters out every fourth one to thin the field evenly. It maps that list to `<svg>` elements, each drawing one wave via `makeWaveOutline` and animating it horizontally with a CSS animation named `wave-drift`. The negative `delay` values start each wave partway through its cycle so the screen is already full of motion on load rather than starting empty.

```
const waves = useMemo(() => {
  const items = []
  let seed = 0
  for (let row = 0; row < ROWS; row++)
    for (let i = 0; i < WAVES_PER_ROW; i++) {
      seed++
      const duration = (14 + ((seed * 1.1) % 10)) * 1.3 * SPEED_FACTOR
      items.push({ seed, topPercent: /* row position + jitter */, sizeCm: /* varied */,
        delay: -(((seed - 1) / total) * duration), duration,
        color: WAVE_COLORS[(seed * 7) % WAVE_COLORS.length] })
    }
  return items.filter((_, idx) => (idx + 1) % 4 !== 0)
}, [])
```

`useMemo(() => {...}, [])` computes the array a single time (the empty dependency array `[]` means "never recompute"). The arithmetic with `seed` and the modulo operator `%` is a cheap way to get varied-but-fixed values without a real random-number generator, so the layout is stable between renders. Every tuning constant — `SPEED_FACTOR`, `HALF_STROKE_WIDTH`, the size range — is named at the top of the file, a sign the visuals were adjusted many times.

8.2 · `ReadingSettingsMenu.tsx`

This file has two responsibilities: it defines the vocabulary of reading options (themes, fonts, size/spacing/width steps, languages) as exported constants, and it renders the pop-over menu that changes them. It also exports a custom hook, `useReadingPrefs`, that other files use to read and persist the settings.

The option catalogs

```
export const THEMES = {
  white: { label: 'White', bg: '#FFFFFF', text: '#1A1A1A', shadow: 'rgba(0,0,0,0.15)' },
  sepia: { label: 'Sepia', bg: '#F4ECD8', text: '#4A3728', shadow: 'rgba(0,0,0,0.15)' },
  /* paper, cream, dark, carbon, shaded, forest ... */
} as const

export const SIZE_STEPS = [0.875, 1, 1.125, 1.25, 1.375, 1.5]
export const WIDTH_STEPS = ['44rem', '50rem', '56rem', '62rem', '68rem']
```

Each theme is an object with a background, text color, and shadow tint. `as const` tells TypeScript to treat these as fixed literal values, which makes the keys (`'white' | 'sepia' | ...`) into a precise type used elsewhere. The steps are plain arrays; the menu stores an *index* into them, not the value, so the controls are simple increment/decrement steppers.

The preferences hook

```
export function useReadingPrefs() {
  const [prefs, setPrefs] = useState<ReadingPrefs>(DEFAULT_PREFS)
  const [loaded, setLoaded] = useState(false)

  useEffect(() => { // on mount: read saved prefs, sanitize, apply
    const raw = localStorage.getItem(STORAGE_KEY)
    if (raw) setPrefs((p) => {
      const merged = { ...p, ...JSON.parse(raw) }
      if (!(merged.font in FONTS)) merged.font = DEFAULT_PREFS.font
      /* ...same guard for theme, lang, and each index... */
      return merged
    })
    setLoaded(true)
  }, [])

  useEffect(() => { if (loaded) localStorage.setItem(STORAGE_KEY, JSON.stringify(prefs)) }, [prefs, loaded])
  return [prefs, setPrefs] as const
}
```

This is a **custom hook** — a function that uses other hooks and packages a reusable behavior. It holds the preferences in state, loads any saved version once on mount, and saves them again whenever they change. The `merged` block is defensive: an old saved setting might name a font or theme that has since been removed, so each field is validated against the current catalogs and reset to the default if unrecognized. This is a real bug fix encoded permanently — an unrecognized font once caused the reader to crash. The `loaded` flag prevents the save effect from firing before the initial load, which would overwrite saved prefs with defaults.

The menu, steppers, and dropdown

The default export is the menu component. It closes itself when you click outside or press Escape (a `useEffect` that adds and later removes document-level event listeners — the cleanup function returned from the effect is what removes them). Inside are a theme swatch grid, a custom `FontDropdown`, three `Stepper` controls (font size, spacing, width), and a language selector. The `Stepper` and `FontDropdown` are small local components defined in the same file because they are used nowhere else. The whole menu is scaled to 1.1× from its top-right corner so it grows leftward while staying pinned to the button — a CSS `transform` detail that keeps the layout from shifting.

8.3 • `TranscriptReader.tsx` — the reading surface

At around 500 lines this is the largest component. It renders the paragraphs and layers three interactive features on top: highlighting, notes, and dictionary lookup. It is worth understanding because it demonstrates how far a single well-structured client component can go.

State

```
const [highlights, setHighlights] = useState<Highlight[]>([])
const [popup, setPopup] = useState<Popup>(null)
const containerRef = useRef<HTMLDivElement>(null)
```

`highlights` is the list of stored highlights for the current transcript and language, loaded from `localStorage` on mount and saved on every change through a `persist` helper. `popup` is a single piece of state describing which floating UI is currently open — a **discriminated union** type: it is either `null`, or an object whose `kind` field is `'actions'` (the Highlight/Dictionary buttons after selecting text), `'note'` (the note editor), or `'define'` (the dictionary result). Storing all pop-ups in one variable with a `kind` tag means only one can be open at a time and the render logic can switch on `kind`.

Turning a selection into a stored range

```
const handleMouseUp = useCallback(() => {
  setTimeout(() => {
    const sel = window.getSelection()
    if (!sel || sel.isCollapsed) return
    const range = sel.getRangeAt(0)
    // find the paragraph elements the selection starts and ends in;
    // for each paragraph it touches, record start/end character offsets
    // using charOffsetWithin(); build a list of per-paragraph "spans"
  }, 0)
}, [])
```

When the mouse is released, the component reads the browser's current text selection. Because a selection can cross paragraph boundaries, it walks every paragraph the selection touches and, using `charOffsetWithin` from `highlights.ts`, converts the visual selection into one or more `{paragraph, start, end}` spans. Those spans are what get stored, so a highlight survives re-rendering and re-opening. The `setTimeout(..., 0)` defers the work by one tick so that any click on an existing highlight is handled first.

Merging overlapping highlights

When you highlight text that overlaps an existing highlight, the two are merged into one rather than stacked. A helper `mergeSpan` finds highlights in the same paragraph whose ranges overlap the new one, computes the union of their start/end, keeps any notes, and replaces them all with a single merged highlight. This keeps the stored data clean and the rendering unambiguous.

Rendering highlighted text

To display a paragraph with highlights, the component cannot just print the text — it must wrap the highlighted character ranges in their own elements. It collects all the highlight boundaries in a paragraph, sorts them into a set of break points, and walks left to right emitting either a plain text run or a `<mark>` element for each slice between break points. Speaker-name prefixes (text before the first colon) are rendered bold-italic. This slice-and-wrap approach is the standard way to render styled ranges over plain text.

The two chronological sections

Below the transcript, the component renders an "All Highlights" section and a "Notes" section, each listing entries in creation order. The notes section shows only highlights that carry a note. Both sections are given HTML anchors (`id="highlights"`, `id="notes"`) so the library's count badges can link directly to them; an effect scrolls to the requested section when the page opens with that anchor in the URL.

Why this lives in one component. Highlighting, notes, and dictionary all operate on the same selection and the same text, and all render pop-ups positioned over that text. Keeping them together avoids passing selection state between components. The cost is a large file; the benefit is that the interactions never fight each other.

9 · The pages

Each `page.tsx` is one screen the user visits. All are client components because all are interactive. They share a visual language — the same header with the underlined wordmark and mint buttons — but each owns its own logic.

9.1 · `layout.tsx` and fonts

The root layout wraps every page. It is one of the few *server* components (no `'use client'`). It loads four fonts through `next/font/google` — Space Grotesk (headline), Anton (display), Source Serif 4 (serif body), Inter (sans body) — and exposes each as a CSS variable on the `<html>` element. Those variables (`--font-headline-family`, etc.) are what the Tailwind font classes point at. It also exports `metadata` (the page title and description used by browsers and link previews) and a `viewport` setting (`width=device-width, initial-scale=1`) that makes the layout render at true size on phones.

```
export const viewport: Viewport = { width: 'device-width', initialScale: 1 }

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={` ${spaceGrotesk.variable} ${sourceSerif.variable} ...`} >
      <body>{children}</body>
    </html>
  )
}
```

`children` is whatever page is being shown; the layout simply places it inside the shared HTML shell. Loading fonts here means they are declared once and available to all pages through the CSS variables.

9.2 · `page.tsx` — the home page

The landing screen: header, a headline, the URL input, and the wave section. Its only real logic is submitting the URL. On submit it does a light validity check (does the text contain `youtube.com` or `youtu.be`), stashes the URL in `sessionStorage`, and navigates to `/transcribing` using the router from `next/navigation`. It also contains a small piece of measurement code: a `useEffect` that, on desktop only, stretches the input row so its right edge lines up with the end of the headline — a purely cosmetic alignment that is disabled below the phone breakpoint to avoid overflow.

```
const handleSubmit = () => {
  if (!url.trim()) { setError('Put a YouTube URL'); return }
  if (!isValidYoutubeUrl(url)) { setError("It doesn't look like a youtube URL"); return }
  setLoading(true)
  sessionStorage.setItem('pending-url', url)
  router.push('/transcribing')
}
```

`sessionStorage` (like `localStorage` but cleared when the tab closes) is used to hand the URL to the next page without putting it in the visible address bar. This is a small, deliberate choice: the loading page has no other way to know which URL to transcribe.

9.3 · `transcribing/page.tsx` — the loading screen

Covered in outline in Chapter 3. The mechanics: it builds a field of waves as mirrored left/right pairs that reveal bottom-up, driven by named constants at the top of the file. It reads `pending-url`, calls `/api/transcribe`, and on success stores the result in `localStorage` and navigates to the reader. The notable engineering detail is the **minimum visible time**:

```
const FILL_COMPLETE_S = (WAVE_ROWS - 1) * ROW_APPEAR_STEP + 0.4
const MIN_VISIBLE_MS = Math.ceil((FILL_COMPLETE_S + 0.6) * 1000)
// ...
const waitForMinimum = async () => {
  const elapsed = Date.now() - startedAt
  if (elapsed < MIN_VISIBLE_MS) await new Promise((r) => setTimeout(r, MIN_VISIBLE_MS - elapsed))
}
```

The minimum is computed from the animation constants, not guessed. Even when the transcript comes back instantly from cache, the page waits so the animation completes at least once — every visit sees the same unhurried fill instead of a flash. A guard ref (`startedRef`) ensures the fetch runs only once even though effects can re-run. On error it shows a message and returns to the home page after a short delay.

9.4 · `transcript/[id]/page.tsx` — the reader

The richest page. It coordinates the reading settings, translation, download, scroll-progress memory, the title's per-line underline, and delegates the actual text to `TranscriptReader`. Key pieces:

- **Instant load.** It reads the local copy synchronously in the initial state (`useState(() => readLocalTranscript(id))`) so the common path — arriving from the loading page — never flashes a spinner. In parallel a `useEffect` tries the database, so a direct link on a fresh device also works.
- **Reading preferences.** It uses `useReadingPrefs` and applies the chosen theme, font, size, spacing, and width to the reading container through inline styles. Dark themes get special treatment — the title underline and word shadows are tuned per surface so they read well on black.
- **Translation.** When the language is not English, it calls `/api/translate`, caches the result in memory and `localStorage`, and shows the translated title and paragraphs. Switching back is instant because both versions are cached.
- **Scroll progress.** A scroll listener writes the percentage to `localStorage`; a one-time effect restores the position on open unless the URL points at a specific section.
- **Download.** A "Download" menu offers plain text (built by joining the paragraphs) and PDF (which triggers the browser's own print-to-PDF against a hidden two-column layout that uses the chosen reading font).
- **Delete.** A button at the end removes the transcript from the database and clears the local copy, then returns to the library.

The per-line title underline is a good example of doing something the platform cannot do directly: CSS underlines cannot have a separate shadow color, so the page measures each wrapped line of the title with `Range.getClientRects()` and draws its own purple bar under each line. This measure-and-draw pattern (used again in the library's hover underline) is how the app achieves effects beyond plain CSS.

9.5 · `library/page.tsx`

Fetches the transcript list from `/api/transcripts` and renders it as boxed rows. For each row it computes, from `localStorage`, the highlight and note counts and the reading percentage, and shows them on the right. Each row's title uses the measure-and-draw underline on hover; a shelf-less trash button deletes the transcript (database plus local cache); the count badges link into the reader's highlight/notes sections. A small local component, `RowTitle`, measures the title's wrapped lines so the hover underline hugs the last line rather than spanning the full width.

9.6 · `feed/page.tsx` and `feed/[videoId]/page.tsx`

`feed/page.tsx` first calls `/api/auth/google/status` and branches: not configured, not connected (show a Connect button linking to the OAuth start route), or connected (fetch `/api/feed` and list the videos). Each video row has a shelf button that sends the video to the library by calling `/api/transcribe` and then removes it from the feed, and a trash button that dismisses it (remembered in `localStorage`). Dismissed videos are filtered out of the list.

`feed/[videoId]/page.tsx` is the watch page: it embeds the YouTube player in an `<iframe>` inside a thick purple bordered box, offers an "Ask AI" button that calls `/api/summary` on demand and renders the TL;DR and bullets, and has a delete button that dismisses the video. The summary is not fetched automatically — only when asked — which keeps the page cheap to open.

10 · Styling: `globals.css` and how Tailwind is wired

One CSS file, `src/app/globals.css`, defines the design tokens, a few global rules, the custom scrollbar, and the wave-animation keyframes. Everything else is Tailwind classes on elements.

```
@import "tailwindcss";

@theme {
  --color-purple: #4E00FF;
  --color-mint: #54FFC9;
  --color-paper: #ECEAE8;
  --color-trash: #F5402E; /* the one deliberate step outside the palette */
  --font-headline: var(--font-headline-family);
  /* ...more colors and font links... */
}

body { @apply bg-cream text-ink font-body antialiased; }
```

`@import "tailwindcss"` pulls in the whole framework. The `@theme` block is Tailwind 4's way of declaring design tokens: each `--color-*` variable automatically becomes usable as a class — `--color-purple` makes `bg-purple`, `text-purple`, `border-purple` all work. The font variables point at the ones the layout defined from the loaded fonts, connecting `font-headline` to Space Grotesk. `@apply` lets a normal CSS rule reuse Tailwind classes, used here to style the `body`.

The rest of the file is a handful of things Tailwind classes cannot express: the global focus outline color, an override so the reading menu shows no focus ring, a custom thin black scrollbar for the font dropdown (with every arrow-button variant hidden, a fix that took several tries across browsers), the print settings, and the `@keyframes` definitions that animate the waves horizontally. Keeping these few rules in one place — and everything else as inline classes — is the Tailwind philosophy: styling lives next to the markup, and the global file holds only what must be global.

11 · Cross-cutting concepts you now understand

Reading the files, the same ideas recurred. Collected here, they are the mental toolkit for the whole project.

- **Async/await and Promises.** Anything touching the network or the database is asynchronous. An `async` function can `await` such work; the result is a `Promise` until it resolves. This is the single most important pattern to be comfortable with.
- **Server versus client.** Route files and the layout run on the server (they may read secrets and the database). Pages and components marked `'use client'` run in the browser (they may use state, effects, and `localStorage`). The boundary is enforced by Next.js.
- **Two-place persistence.** Shared, durable data (transcripts) goes to Postgres. Personal, lightweight data (highlights, notes, prefs, progress, dismissals) goes to `localStorage`. This split is why the app needs no user accounts yet still remembers things.
- **Cache before you spend.** The transcribe route checks the database first; the summary route reuses a saved transcript; translations are cached per language. External calls cost money and time, so results are stored and reused.
- **Fail safe, fail loud.** The AI cleanup falls back to heuristic paragraphs; translation refuses to return misaligned text; every `localStorage` read has a default. Where a wrong answer would be worse than none, the code returns none.
- **Measure, don't hardcode.** Underlines, input widths, and the loading-time minimum are computed from the live page or from named constants rather than guessed, so they stay correct when content changes.
- **Defensive parsing.** Optional chaining (`?.`), nullish coalescing (`??`), and `try/catch` appear wherever data comes from outside — because outside data cannot be trusted to have the expected shape.

12 · Architectural decisions, restated

Pulled together, the choices that shaped the codebase, each with its trade-off.

Decision	Why	Cost
One Next.js app, no separate backend	Fewer moving parts; secrets stay in server code; one deploy.	Backend logic is bound to Next.js conventions.
Transcripts in Postgres, personal data in localStorage	No login needed; instant local reads; sharable transcripts.	Highlights/notes/progress do not follow you across devices.
Transcribe once, cache forever	Transcription is the only slow, paid step.	Stale metadata if a video's title changes; no re-clean without deleting.
AI cleanup as an optional stage	The app still produces output if Gemini fails.	Fallback text is rougher.
Prompt-based correction list	Fixes are easy to add; no code change.	The prompt grows; the model may still miss cases.
Single-user Google connection (one cookie)	Avoids building accounts and OAuth verification.	Not usable by multiple people as-is.
Only four runtime dependencies	Small, auditable, fast to build.	More code written by hand (OAuth, wave math, dropdowns).
Tailwind with many arbitrary values	Pixel-precise design controlled inline.	Verbose class strings; design lives in markup.

13 · The limits of the current application

An honest inventory of what the app does not do, or does narrowly. None of these are bugs; they are boundaries of the current design.

- **Single user.** There are no end-user accounts. The Google/feed connection stores one refresh token per browser. Two people cannot each have their own feed on the same deployment.
- **Personal data is device-bound.** Highlights, notes, reading preferences, scroll progress, and feed dismissals live in `localStorage`. Clearing browser data erases them; a second device starts blank.
- **Captions required.** Everything depends on a video having captions. No captions means no transcript and no summary.
- **English-first.** Captions are requested in English; translation is to French or Spanish only; the dictionary is reliable mainly for English.
- **Transcription time limit.** Very long videos can approach the five-minute function ceiling. There is no background job system — the request must finish within that window.
- **Word-level timing is lost.** After cleanup, each paragraph's `startMs` is `0`; the reader cannot jump to a moment in the video from a paragraph.
- **Feed is quota-bound and capped.** The 30-day window, ~100-channel and Shorts limits exist to stay within YouTube's daily API quota; a very large subscription list is truncated.
- **No automated tests.** Correctness is verified by building and by manual/visual checks, not by a test suite.
- **Metadata can go stale.** Because transcripts are cached forever, a later title change on YouTube is not reflected unless the transcript is deleted and remade.
- **Cost scales with new videos.** Every first-time transcription spends Supadata and Gemini credits; there is no rate limiting or per-user budget.

14 · What scaling would require

If this went from a personal tool to something many people use, here is roughly what each step would demand, in increasing order of effort.

Make personal data follow the user

Introduce real user accounts (sign in with Google for everyone, not just the feed), and move highlights, notes, preferences, and progress from `localStorage` into database tables keyed by user ID. This is the single change that unlocks multi-device use and multi-user use at once. It requires a per-user token store (replacing the single cookie) and Google's OAuth verification review so strangers can connect.

Move transcription off the request path

Replace the synchronous "wait five minutes" model with a queue: the request enqueues a job and returns immediately; a background worker (a scheduled function or a dedicated service) does the caption fetch and cleanup and writes the result; the client polls or is notified. This removes the time-limit ceiling, lets long videos finish, and enables the auto-queue feed idea (new uploads transcribed automatically under rules).

Control cost and abuse

Add per-user rate limits and a spending cap, since each new transcription costs money. Cache negative results (videos known to have no captions) to avoid repeat work. Consider a cheaper model or a two-tier approach (cheap pass, expensive pass only when asked).

Harden data and operations

Add a real migration system instead of the lazy `ALTER TABLE` in `db.ts`; add indexes as the transcripts table grows; add an automated test suite around the parsing and prompt-adjacent code so prompt changes do not silently regress; add monitoring for the external services so failures are visible.

Broaden reach

Support non-English captions end to end, more translation targets, and per-video language detection. Store word-level timings so the reader can link paragraphs to moments in the embedded player. Each of these is additive and does not require rearchitecting.

The order matters. Accounts and a job queue are foundational; cost controls and hardening follow; reach features are the easy, incremental layer on top. The current code is structured so these can be added without rewriting the core: storage is already isolated in `db.ts`, external calls in `lib/`, and the UI reads through small, replaceable functions.

15 · Glossary

Term	Meaning
API endpoint / route	A URL on the server that returns data (JSON), not a page. Here, the <code>route.ts</code> files.
async / await / Promise	Machinery for code that waits on slow work (network, database) without freezing.
bundler	A tool that combines and transforms source files into what the browser downloads. Next.js includes one.
client component	A React component that runs in the browser; marked <code>'use client'</code> .
cookie	A small value the browser stores and sends back to the server with each request. Used here for the Google token.
CRUD	Create, Read, Update, Delete — the basic database operations.
destructuring	Pulling fields out of an object or array by name/position: <code>const { url } = body</code> .
environment variable	A named secret configured in the deployment, read with <code>process.env</code> , never in the code.
fetch	The built-in function for making an HTTP request.
hook	A React function starting with <code>use</code> that adds memory or behavior to a component.
HTTP status code	A number signaling the outcome: 200 ok, 400 bad request, 401 unauthorized, 404 not found, 500 server error.
JSON	A text format for data (objects, arrays, strings, numbers). The lingua franca between the pieces here.
JSX	HTML-like syntax inside JavaScript that describes what a component renders.
localStorage / sessionStorage	Browser key–value stores for strings. <code>local</code> persists; <code>session</code> clears with the tab.
OAuth	The standard flow for letting an app access your account on another service without your password.
optional chaining <code>?.</code>	Read a nested field only if the parent exists, else <code>undefined</code> .
nullish coalescing <code>??</code>	Use the left value unless it is null/undefined, then the right.
regular expression	A pattern for matching text, written between slashes: <code>/#shorts/i</code> .
server component	A React component that renders on the server to HTML; the default in the App Router.
SQL	The language for querying a relational database.
SVG	A vector-graphics format; shapes described by path strings. Used for the waves and icons.
tagged template literal	A backtick string handled by a function, e.g. <code>sql`SELECT ...`</code> , which safely inserts values.
ternary <code>? :</code>	An inline if/else that produces a value.
type / interface	A TypeScript description of a value's shape, checked before the code runs.
union type	A value that may be one of several types: <code>string null</code> .

upsert	Insert a row, or update it if it already exists (<code>ON CONFLICT DO UPDATE</code>).
--------	---

16 · Syntax appendix

A compact reference for the syntax that appears most, gathered so you can look it up while reading the code.

Variables and functions

```
const x = 5           // a value that will not be reassigned
let y = 10            // a value that may change
function f(a: number): number { return a + 1 } // named function, typed
const g = (a: number) => a + 1                 // arrow function (same thing)
const h = async () => { await something() }     // async arrow function
```

Objects, arrays, destructuring

```
const o = { name: 'A', age: 3 } // object
const { name } = o              // pull out `name`
const a = [1, 2, 3]             // array
const [first, second] = a       // pull out by position
const copy = { ...o, age: 4 }   // spread: copy o, override age
const bigger = [...a, 4]        // spread into a new array
```

Array transforms (used constantly)

```
a.map(x => x * 2)           // [2,4,6] - one array to another
a.filter(x => x > 1)         // [2,3]   - keep some
a.sort((p, q) => p - q)      // ascending; q - p for descending
a.join(' ')                 // '1 2 3' - array to string
'x y'.split(' ')            // ['x','y'] - string to array
```

Conditionals and safety operators

```
cond ? whenTrue : whenFalse // ternary
value ?? fallback            // use value unless null/undefined
obj?.field?.sub              // read only if each step exists
!!value                      // coerce to strict true/false
```

TypeScript types

```
interface Segment { text: string; startMs: number } // a named object shape
type Id = string | null                             // a union
function get(): Promise<Segment[] | null> { ... }    // returns a promise of array-or-null
const t = value as const                            // treat literals as fixed types
```

React essentials

```

const [n, setN] = useState(0) // state
useEffect(() => { /* run after render */ }, [dep]) // effect + dependencies
useEffect(() => { const off = subscribe(); return () => off() }, []) // cleanup on
unmount
const ref = useRef<HTMLDivElement>(null) // a handle to an element
const v = useMemo(() => compute(x), [x]) // memoized value
<div className="flex gap-2" onClick={handle}>{child}</div> // JSX with a class and
handler
{items.map(it => <li key={it.id}>{it.text}</li>)} // rendering a list (each
needs a key)

```

Server route shape

```

export async function POST(req: Request) {
  const body = await req.json()
  // ...do work...
  return NextResponse.json({ ok: true }, { status: 200 })
}

```

End of document. You have now seen every source file in the project at least in outline, the reasoning behind the main decisions, the current limits, and the path to scaling. Re-reading any single chapter against the open file it describes is the fastest way to make the syntax feel routine. The codebase is small enough that, with this guide beside it, nothing in it needs to remain mysterious.